National University of Computer & Emerging Sciences, Karachi
Computer Science Department
Spring 2026, Lab Manual – 08

Course Code: CL-1004	**Course : Object Oriented Programming Lab**
Instructor :	**Muhammad Owais**

LAB - 8

Types of Inheritance & Interface

Types of Inheritance:

1 – Single level

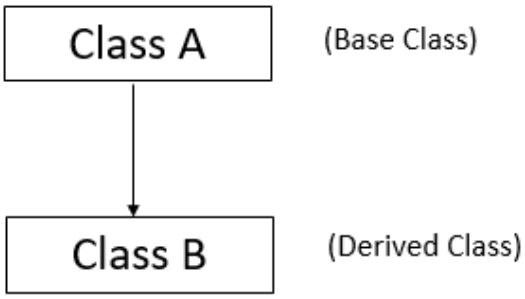
2 – Multilevel Inheritance

3 – Hierarchical Inheritance

4 – Multiple Inheritance

5 – Hybrid Inheritance

1 – Single level

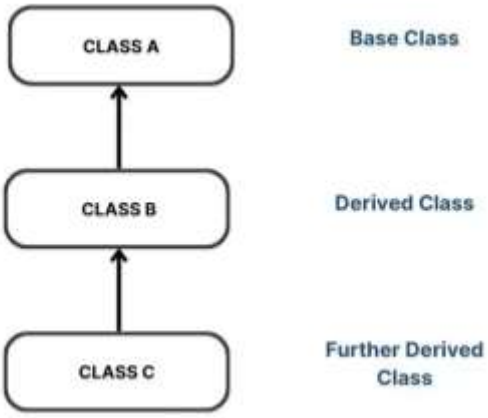


Single inheritance is a type of inheritance in which one subclass inherits the properties and methods of a single superclass.

In this type, there is only one parent class and one child class.

```
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

public class Main {
    public static void main(String[] args) {

        Dog d = new Dog();

        d.eat();   // inherited method
        d.bark();  // child class method
    }
}
```

2 – Multilevel Inheritance



Multilevel inheritance is a type of inheritance in which a class inherits from another class, and that class becomes the parent of another class.

In this type, inheritance forms a chain or multiple levels of classes.

Example:

Person → Employee → Manager

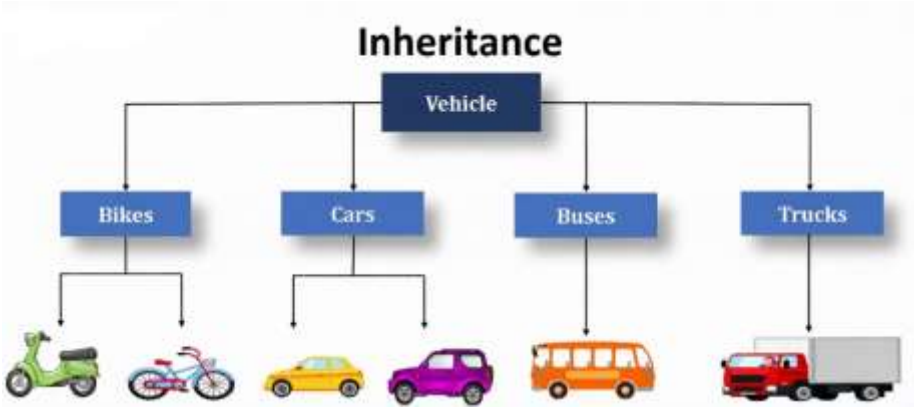
- Employee inherits from Person
- Manager inherits from Employee

```
class Person {
    void showPerson() {
        System.out.println("I am a person");
    }
}
class Employee extends Person {
    void showEmployee() {
        System.out.println("I am an employee");
    }
}
class Manager extends Employee {
    void showManager() {
        System.out.println("I am a manager");
    }
}

public class Main {
    public static void main(String[] args) {

        Manager m = new Manager();

        m.showPerson();
        m.showEmployee();
        m.showManager();
    }
}
```

3 – Hierarchical Inheritance

Hierarchical inheritance is a type of inheritance in which multiple subclasses inherit from a single superclass.

In this type, the same parent class is shared by more than one child class.



Example:

A Vehicle is a parent class, and Car and Bike are child classes that inherit the properties of Vehicle.

```
class Vehicle {
    void start() {
        System.out.println("Vehicle starts");
    }
}
class Car extends Vehicle {
    void drive() {
        System.out.println("Car is driving");
    }
}
class Bike extends Vehicle {
    void ride() {
        System.out.println("Bike is riding");
    }
}

public class Main {
    public static void main(String[] args) {

        Car c = new Car();
        Bike b = new Bike();

        c.start();
        c.drive();

        b.start();
        b.ride();
    }
}
```

4 – Multiple Inheritance

Multiple inheritance is a type of inheritance in which one class inherits the properties and methods of more than one parent class.

In other words, a single child class has multiple parent classes.

Java does not support multiple inheritance using classes because it may create the Diamond Problem (ambiguity when two parent classes have the same method).

Example:

```java
class A {
    void show() {
        System.out.println("Class A");
    }
}

class B {
    void show() {
        System.out.println("Class B");
    }
}

class C extends A, B {   // ❌ Not allowed in Java
}
```

Java prevents this situation because the compiler would not know which show() method to call (A or B).

Java supports multiple inheritance using interfaces.

Interfaces:

An **interface** in Java is a blueprint of a class. It has static constants and abstract methods. A difference between abstract classes and interface is that there can be concrete methods in abstract classes whereas interface cannot contain any.

How to create an interface:

An interface is declared by using the interface keyword. It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default. A class that implements an interface must implement all the methods declared in the interface.

Syntax:

interface *<interface_name>{*

// declare constant fields
// declare methods that are abstract
// by default.
}

Example: *(Declaring an interface)*

```
interface printable{
    public void print();

}
```

Example: *(Implementing interface in class)*

```
class Test implements printable{
    public void print(){
        System.out.println(" I was declared in interface and I am implemented in class");
    }
}
```

Points to Remember: We can create static methods in interfaces. But we must define their implementation inside the interface.

Example:

```
interface printable{
public static void print(){
    System.out.println("I am static method of Interface");}
}
```

Points to Remember: When a class implements an interface, we can save the object of that class in the interface variable.

Example:

```
public class Main
{
    public static void main(String[] args) {
        printable obj = new Test();
        obj.print();

    }
}
```

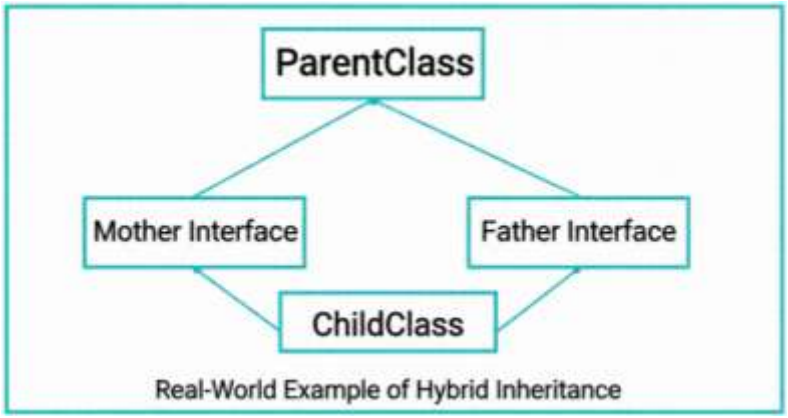
```
I was declared in interface and
I am implemented in class
```

Multiple Inheritance Using Interfaces

```
interface Camera {
    int resolution = 108;

    void takePhoto();
}

interface MusicPlayer {
    int volumeLevel = 10;

    void playMusic();
}

class SmartPhone implements Camera, MusicPlayer {

    public void takePhoto() {
        System.out.println("Taking photo with resolution: " + resolution + " MP");
    }

    public void playMusic() {
        System.out.println("Playing music at volume level: " + volumeLevel);
    }
}

public class Main {
    public static void main(String[] args) {

        SmartPhone s = new SmartPhone();
        s.takePhoto();
        s.playMusic();
    }
}
```

5 – Hybrid Inheritance

Hybrid inheritance is a type of inheritance that is formed by combining two or more types of inheritance, such as Hierarchical and Multiple inheritance.



Java does not support hybrid inheritance using classes alone, because it may create ambiguity (diamond problem).

However, it can be implemented using a combination of classes and interfaces.

Hybrid Inheritance Using Interfaces

```java
interface Device {
    String brand = "Samsung";

    void powerOn();
}

interface Camera extends Device {
    int resolution = 108;

    void takePhoto();
}

interface MusicPlayer extends Device {
    int volume = 10;

    void playMusic();
}

class SmartPhone implements Camera, MusicPlayer {

    public void powerOn() {
        System.out.println("Device " + brand + " is powered on");
    }

    public void takePhoto() {
        System.out.println("Taking photo with resolution " + resolution + " MP");
    }

    public void playMusic() {
        System.out.println("Playing music at volume " + volume);
    }

     void call() {
         System.out.println("SmartPhone making a call");
     }
 }

 public class Main {
     public static void main(String[] args) {

         SmartPhone s = new SmartPhone();

         s.powerOn();
         s.takePhoto();
         s.playMusic();
         s.call();
     }
 }
```

Output:

```
Device Samsung is powered on
Taking photo with resolution 108 MP
Playing music at volume 10
SmartPhone making a call
```